

The Magic of Trapdoor Hash Function

Emanuele Giunta¹

ETH Zürich

Disclaimer(s)

1. This is not my own work, just a research area I enjoy.
2. Goal: give an overview of this primitive and how to use it.

Trapdoor Hash Functions: Setting



- Alice only learns $f(x)$
- Bob learns nothing about f

Trapdoor Hash Functions: Primitive

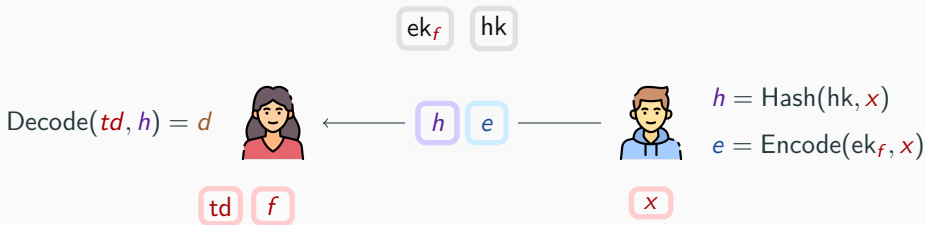
Setup(1^λ)

KeyGen(hk, f)

Hash(hk, x)

Encode(ek, x)

Decode(td, h)



Correctness*

$$e = d \oplus f(x)$$

Compactness

$$|h| = \text{poly}(\lambda)$$

Rate-1

$$|e| = |f(x)|$$

Input Privacy*

Only $f(x)$ leaked

Function Hiding

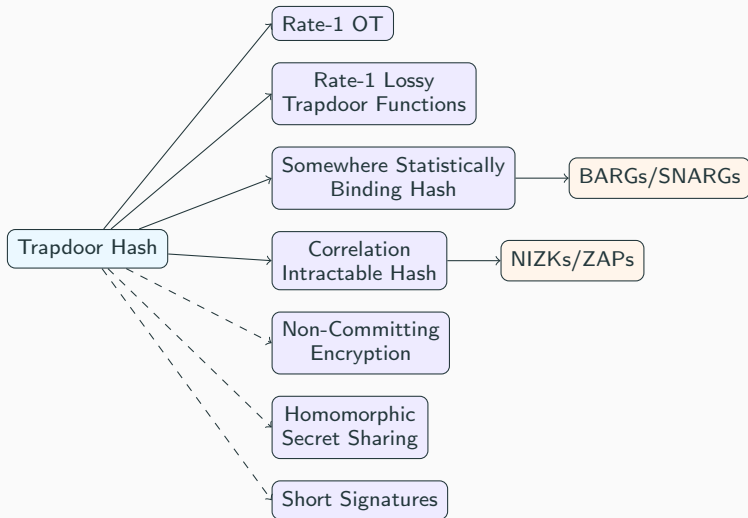
$$ek_f \approx ek_g$$

TDH for Linear Functions

TDH supporting **linear functions** are known from:

- ✓ [DGI⁺19] Constructions under **LWE, DCR, QR, DDH**
- ! DDH, DCR and LWE with polynomial noise-to-modulus ratio suffer a $1/\text{poly}(\lambda)$ correctness error
- ✓ [AMR 25] Construction from low-noise **LPN**
- ! $1/\text{poly}(\lambda)$ -error and low compactness
- ✗ No construction from Group Actions/Isogenies, MQ, Search Assumptions.

Applications



Applications In this talk

1. Semi-Honest Oblivious Transfer
2. Somewhere Statistically Binding Hash
3. Lossy Public Key Encryption
4. Correlation Intractable Hash

! All primitives above are **Rate-1**.

Oblivious Transfer

Oblivious Transfer



Sender Privacy

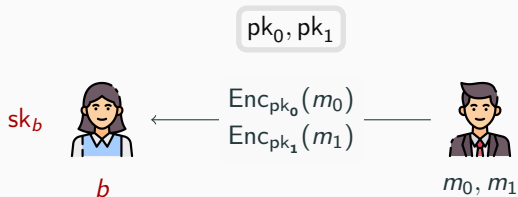
No leakage on m_{1-b}

Receiver Privacy

No leakage on b

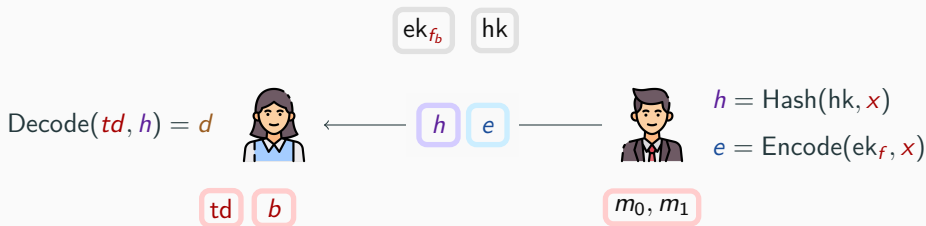
Rate-1/2 OT

Prior to TDH: **two-keys paradigm**



Inherently at best **Rate-1/2**.

OT from Linear Trapdoor Hash Functions



Function
 $f_b(m_0, m_1) = m_b$

Correctness
 $e = d \oplus m_b$

Rate ≈ 1
 $|h| + |e| \approx |m_b|$

Batched OT from Linear Trapdoor Hash Functions



Function

$$f_{\vec{b}}(\vec{m}_0, \vec{m}_1) = (m_{i,b_i})_{i=1}^n$$

Correctness

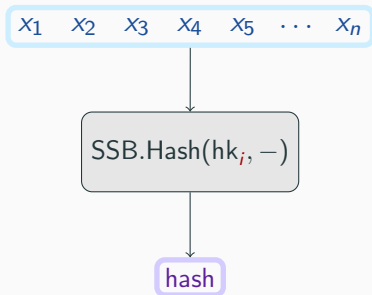
$$e = d \oplus (m_{i,b_i})_{i=1}^n$$

Rate ≈ 1

$$|h| + |e| \approx \sum_i |m_{i,b_i}|$$

Somewhere Statistically Binding Hash

SSB Hash



Somewhere Statistically Binding

Two **inputs** differing at the i -th position have different **hash value**

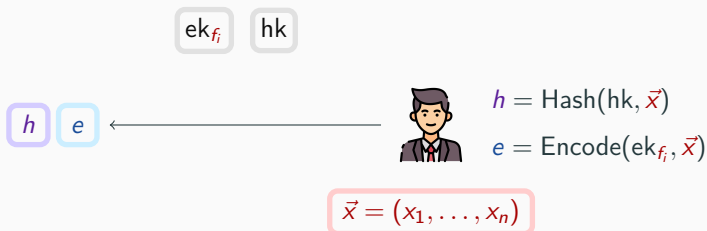
Index Hiding

$$hk_i \approx hk_j$$

Rate 1

$$|\text{hash}| \approx |x_i|$$

SSB Hash from Linear Trapdoor Hash



Function
 f_i the i -th projection
 $f_i(\vec{x}) = x_i$

SSB
 $e = d \oplus x_i$
(a reduction can extract x_i)

Rate ≈ 1
 $|h| + |e| \approx |x_i|$

Lossy Public Key Encryption

Lossy Public Key Encryption

Normal Mode

$(pk_1, sk_1) \leftarrow \text{GenInj}$

$m \mapsto \text{Enc}(pk_1, m; r)$
is injective/decryptable

$\approx^c$

Lossy Mode

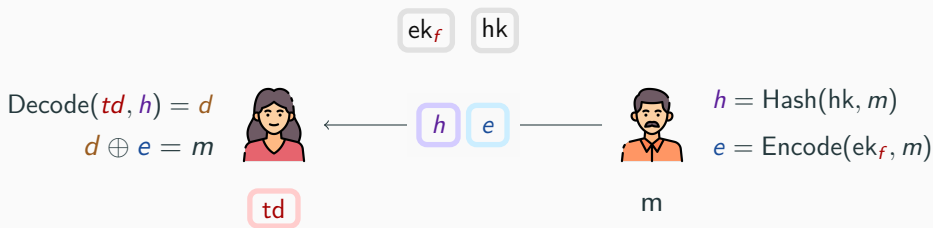
$pk_0 \leftarrow \text{GenLoss}$

$\log |\text{Im Enc}(pk_0, -; r)| \ll |m|$
 $\text{Enc}(pk_0, m)$ independent of m

✓ Leakage Resilience

✓ Tight Security

Encryption from Linear Trapdoor Hash



Function
 $f(m) = m$
the identity

Correctness
 $e = d \oplus m$

Rate-1
 $|h| + |e| \approx |m|$
ciphertext is (h, e)

Encryption from Linear Trapdoor Hash

E.GenInj(1^λ):

$hk = H.Setup(1^\lambda, 1^n), (\text{td}, ek_I) = H.KeyGen(hk, I)$

$pk = (hk, ek_I), sk = \text{td}$

E.GenLoss(1^λ):

$hk = H.Setup(1^\lambda, 1^n), (\text{td}, ek_O) = H.KeyGen(hk, O)$

$pk = (hk, ek_I)$

E.Encrypt(pk, m):

$h = H.Hash(hk, m), e = H.Encode(ek_I, m)$

$ct = (h, e)$

E.Decrypt(pk, ct):

$d = H.Decode(\text{td}, h)$

$m = e \oplus d$

Normal: $e = d \oplus m$

Lossy: $e = d$

Correlation Intractability

Correlation Intractable Hash

$(\text{Gen}, \text{Hash})$ is **correlation intractable** for $\mathcal{R}$ if given key $K \leftarrow \text{Gen}(1^\lambda)$ it is hard to find $\vec{x}$ s.t.

$$(\vec{x}, \vec{y}) \in \mathcal{R} \quad \wedge \quad y_i = \text{Hash}(K, x_i).$$

Examples

- **Collision Resistance.** $\mathcal{R}_{\text{eq}}$ of (x_1, x_2, y_1, y_2) such that $x_1 \neq x_2$ and $y_1 = y_2$.
- **Near Collision Resistance.** $\mathcal{R}_{\text{near}}$ of (x_1, x_2, y_1, y_2) such that $x_1 \neq x_2$ and $\Delta(y_1, y_2) \leq \delta$.
- **Search Relations.** $\mathcal{R}_C$ of (x, y) such that $y = C(x)$.

Observation 1

CI for a **single** search relation $\mathcal{R}_C$ is easy:

$$x \mapsto C(x) \oplus r \quad \text{for some } r \neq 0$$

... However, we aim to support search relations $\mathcal{R}_C$ for **all** $C \in \mathcal{C}$ some circuit family $\mathcal{C}$

Observation 2

For a trapdoor hash, the encoding function for ek_C is shifted by a **very lossy function** from C .

$$e(x) = d(x) \oplus C(x)$$

$$d(x) = \text{Decode}(td, \text{Hash}(hk, x))$$

$$\log |\text{Im } d(x)| \leq \log |\text{Im Hash}(hk, -)|$$

CI from Trapdoor Hash Function

! We assume $C : \{0, 1\}^n \rightarrow \{0, 1\}^m$ has **large** output length $m \gg \lambda$.

CIH.Gen(1^λ):

$hk = H.Setup(1^\lambda, 1^n)$, $(\text{td}, ek_C) = H.KeyGen(hk, C)$, $r \xleftarrow{\$} \{0, 1\}^m$

Return $K = (ek_C, r)$

CIH.Hash(K, x):

Return $\text{hash} = H.Encode(ek_C, x) \oplus r$

Corr. Intractability for C

$$e(x) \oplus r = C(x)$$

$$\Rightarrow d(x) \oplus C(x) \oplus r = C(x)$$

$$\Rightarrow r = d(x) \quad (\text{very unlikely})$$

Corr. Intractability for $C' \neq C$

Use function hiding to switch

$$ek_C \approx ek_{C'}$$

Conclusion

Conclusion

- ✔ TDHs are a versatile tool to build **succinct** primitives.
- ❓ Not known from some important assumptions (e.g. isogenies)
- ❓ Without LWE, not known to support expressive circuit families

Thanks for your attention!

